

Instance Segmentation Pipeline for Gas Bubble Multiphase Images (PyTorch)

This guide outlines a modular pipeline to train an instance segmentation model (e.g. Mask R-CNN) on custom gas-bubble images using PyTorch. It covers data annotation and conversion, data loaders with augmentations, a Mask R-CNN example, how to plug in other models, hyperparameter/configuration management, evaluation, experiment tracking, and code structure. Example code snippets and links to open-source resources (GitHub, docs) are provided for reference.

1. Setup and Dependencies

- **Environment:** Use Python 3 with PyTorch (and torchvision) installed. A GPU is recommended for training.
- **Install COCO tools:** Many detection/segmentation tasks use the COCO format. Install the COCO API (`pycocotools`) for mask handling and evaluation. For example:

```
pip install pycocotools
```

(Arpit Aggarwal's Mask R-CNN demo on a custom dataset also installs COCO API ¹.)

- **Additional libraries:** Common packages include `numpy`, `torchvision`, and possibly `albumentations` or `imgaug` for augmentations. For logging, install Weights & Biases (`wandb`), MLflow, and TensorBoard (`tensorboard`) if you plan to use them (see Tracking tools below).

2. Data Annotation and Format Conversion

Gas-bubble images need instance-level annotations (each bubble labeled separately). If you have **brush annotations** or polygons from tools like LabelMe or CVAT, convert them to a standard format such as COCO JSON. Two common routes are:

- **LabelMask → COCO:** If each image has a single mask (binary or multi-color) per object, you can programmatically convert masks to COCO format. TorchVision's tutorial on Penn-Fudan shows how to split a color mask into binary masks and extract bounding boxes ². For example:

```
from torchvision.io import read_image
import torch
mask = read_image(mask_path) # e.g. color-coded mask image
obj_ids = torch.unique(mask)[1:] # drop background id 0
masks = (mask == obj_ids[:, None, None]).to(torch.uint8)
```

These `masks` (shape `[num_objs, H, W]`) and their bounding boxes can then be packaged into a COCO-style dictionary.

- **LabelMe/CVAT JSON → COCO JSON:** If you used LabelMe or similar tools, you can use converters. For instance, the `labelme2coco` utility converts LabelMe JSONs to COCO format in

one step ³. Roboflow and others also document JSON conversions. Pascal VOC (XML or mask PNG) is older; COCO is recommended for modern instance segmentation.

Key points on COCO format: it uses per-object polygon (or RLE) masks plus class labels. Make sure your converter sets "segmentation", "bbox", "category_id", etc., correctly. The TorchVision tutorial and GitHub examples assume images and masks are aligned in folders and load them per index ⁴

⁵.

3. Data Loading and Augmentation

Write a **PyTorch Dataset** class that loads images and annotations. For example, akin to TorchVision's PennFudan demo:

```
class BubbleDataset(torch.utils.data.Dataset):
    def __init__(self, root, transforms=None):
        self.root = root
        self.imgs = sorted(os.listdir(os.path.join(root, "images")))
        self.masks = sorted(os.listdir(os.path.join(root, "masks")))
        self.transforms = transforms

    def __getitem__(self, idx):
        img = read_image(os.path.join(self.root, "images", self.imgs[idx]))
        mask = read_image(os.path.join(self.root, "masks", self.masks[idx]))
        obj_ids = torch.unique(mask)[1:] # skip background
        masks = (mask == obj_ids[:, None, None]).to(torch.uint8)
        boxes = torchvision.ops.masks_to_boxes(masks) # get bounding boxes

        target = {
            "boxes": boxes,
            "labels": torch.ones((len(obj_ids),), dtype=torch.int64),
            "masks": masks,
            "image_id": torch.tensor([idx]),
            "area": (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:,
0]),
            "iscrowd": torch.zeros((len(obj_ids),), dtype=torch.int64)
        }
        if self.transforms:
            img, target = self.transforms(img, target)
        return img, target

    def __len__(self):
        return len(self.imgs)
```

Then create a **DataLoader** with a custom collate function to batch variable-size images/targets (as in the tutorial, they use `utils.collate_fn`) ⁶. Example:

```
from torch.utils.data import DataLoader
def collate_fn(batch):
    return tuple(zip(*batch))
```

```
data_loader = DataLoader(dataset, batch_size=4, shuffle=True,
collate_fn=collate_fn)
data_loader_test = DataLoader(dataset_test, batch_size=2, shuffle=False,
collate_fn=collate_fn)
```

Data Augmentation: Apply transforms to images and targets. You can use `torchvision.transforms` (e.g. `RandomHorizontalFlip`, `ColorJitter`) inside the `transforms` passed to the dataset. For instance:

```
from torchvision.transforms import functional as F
def get_transform(train):
    transforms = []
    if train:
        transforms.append(RandomHorizontalFlip(0.5))
        transforms.append(ColorJitter(...))
    return Compose(transforms)

dataset = BubbleDataset("data/Bubbles", transforms=get_transform(train=True))
```

Popular libraries like [Albumentations](#) can also do instance segmentation augmentation (random crops, elastic deformations, etc.). Document which augmentations you use, since they can significantly affect results.

4. Baseline Model: Mask R-CNN

Start with a **Mask R-CNN** from TorchVision as a baseline. TorchVision provides a pre-built `maskrcnn_resnet50_fpn` model. You can load pretrained COCO weights and then reset the classifier/mask heads for your classes:

```
import torchvision
# Suppose num_classes = 2 (background + bubble)
model = torchvision.models.detection.maskrcnn_resnet50_fpn(
    weights='COCO',
    # or use MaskRCNN_ResNet50_FPN_Weights.DEFAULT in newer versions
    num_classes=num_classes # include background as one class
)
```

This model uses a ResNet-50 backbone with FPN, pretrained on COCO ⁷ ⁸. Move it to GPU with `model.to(device)` and set it to training mode. For a custom dataset, you may fine-tune from COCO weights (fast) or train from scratch if you have enough data. Arpit Aggarwal's Mask R-CNN on Penn-Fudan uses a similar approach ⁷.

TorchVision's `example` wraps model creation in a helper function (`get_model_instance_segmentation(num_classes)`) to easily reuse it ⁹. You could similarly define a factory function that chooses the model based on config (see **Extensions** below).

5. Training Loop

Implement a standard training loop. The official detection tutorial separates training into `train_one_epoch()` and `evaluate()` functions (you can adapt those or write your own) ¹⁰. A simplified structure:

```
import torch
from torch.optim import SGD
from torch.optim.lr_scheduler import StepLR
from engine import train_one_epoch, evaluate # from TorchVision tutorials

device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')
model.to(device)

# Optimizer and scheduler (example)
params = [p for p in model.parameters() if p.requires_grad]
optimizer = SGD(params, lr=0.005, momentum=0.9, weight_decay=5e-4)
lr_scheduler = StepLR(optimizer, step_size=3, gamma=0.1)

num_epochs = 20
for epoch in range(num_epochs):
    train_one_epoch(model, optimizer, data_loader, device, epoch,
    print_freq=50)
    lr_scheduler.step()
    evaluate(model, data_loader_test, device=device) # prints mAP and losses
```

¹⁰

Here, `train_one_epoch()` handles forward/backprop for each batch, and `evaluate()` uses COCO metrics. The function `evaluate()` (from TorchVision's [engine.py](#)) computes box/mask mAP on the test set. By calling `evaluate()` each epoch, you can monitor performance.

Hyperparameters: You control learning rate, number of epochs, batch size, etc. These should ideally come from your config/CLI (see next section). Adjust them based on validation performance. A common setup for Mask R-CNN is SGD with LR ~0.005–0.02, momentum 0.9, weight decay ~1e-4, training 10–50 epochs. For small datasets, use lower LR and fewer epochs.

6. Configuration and Hyperparameters

Make training configurable. Use a **command-line interface (argparse)** or **config file (YAML/JSON)** to set hyperparameters and paths. For example, with argparse:

```
import argparse
parser = argparse.ArgumentParser(description="Train instance segmentation")
parser.add_argument("--epochs", type=int, default=20)
parser.add_argument("--lr", type=float, default=0.005)
parser.add_argument("--batch_size", type=int, default=4)
parser.add_argument("--model", type=str, default="maskrcnn")
```

```

parser.add_argument("--data_path", type=str, default="data/Bubbles")
# ... add more args as needed
args = parser.parse_args()

```

Then use `args.lr`, `args.epochs`, etc. in your code. As PyTorch Lightning's docs note, it's common to layer arguments (trainer args, model args, data args) for clarity ¹¹ ¹². For example, after `parser = ArgumentParser()`, you can group model-specific and data-specific args.

Alternatively, use a config file (e.g. YAML). For instance:

```

# config.yaml
model: maskrcnn
num_classes: 2
learning_rate: 0.005
momentum: 0.9
batch_size: 4
num_epochs: 20
data_root: data/Bubbles

```

Load it in Python with PyYAML or OmegaConf:

```

import yaml
with open("config.yaml") as f:
    cfg = yaml.safe_load(f)
# Access cfg["learning_rate"], cfg["num_epochs"], etc.

```

Tools like **Hydra** or **PyTorch Lightning's CLI** provide advanced config management. For example, Lightning makes it easy to parse arguments in `LightningModule` and call your script with flags like `--learning_rate 0.01` ¹¹. MMDetection uses a full-featured config system where you can override any option via `--cfg-options` on the command line ¹³.

7. Extending to Other Models

Design your code so you can swap models. For example, map string names to constructors:

```

import torchvision
from yolact import YolactModel # hypothetical import

model_factories = {
    "maskrcnn": lambda num_classes:
        torchvision.models.detection.maskrcnn_resnet50_fpn(weights='COCO',
        num_classes=num_classes),
    "fasterrcnn": lambda num_classes:
        torchvision.models.detection.fasterrcnn_resnet50_fpn(weights='COCO',
        num_classes=num_classes),
    "retinanet": lambda num_classes:

```

```

torchvision.models.detection.retinanet_resnet50_fpn(weights='COCO',
num_classes=num_classes),
    "yolact": lambda num_classes: YolactModel(num_classes),
    # etc.
}
model = model_factories[args.model](num_classes=args.num_classes)

```

TorchVision provides other detectors (Faster R-CNN, RetinaNet) which can often be adapted for segmentation (e.g. by predicting masks). For dedicated instance-seg models beyond TorchVision, consider **Detectron2** or **MMDetection**: these frameworks have many architectures (Cascade Mask R-CNN, SOLO, YOLACT, etc.) ready to use. For example, Facebook's Detectron2 is a popular PyTorch library for segmentation ¹⁴, and OpenMMLab's MMDetection offers a modular toolbox for detection/segmentation ¹⁵. You can integrate them by writing similar training scripts (Detectron2 has a simple `DefaultTrainer`), or by exporting your data in COCO format and using their tools.

As an example, the **YOLACT** repo (by D. Bolya) provides a real-time segmentation model (PyTorch-based) which you could plug in ¹⁶. The key is to abstract model creation so you can switch architectures by name, and ensure your data loader and loss functions match the chosen model's API.

8. Training and Evaluation Configuration

- **Optimizer and LR scheduling:** In the training loop, use optimizers like SGD or Adam. Example: SGD with momentum and a StepLR (as above). Expose LR and scheduler parameters via config/CLI.
- **Epoch control:** Use `args.epochs` (or config) to set the number of epochs. Optionally, implement early stopping or use validation metrics to save best checkpoints.
- **Loss monitoring:** Collect losses and metrics each iteration. Use these for logging and adjusting hyperparameters. TorchVision's `train_one_epoch()` returns a log of losses.
- **Evaluation metrics:** For instance segmentation, common metrics are COCO mAP (Average Precision) for bounding boxes and masks. The TorchVision `evaluate()` prints COCO-style AP_{50} , AP_{75} , etc ¹⁰. You may also compute custom metrics (e.g. mean IoU for bubbles). Ensure that your "evaluate" function can work in inference mode (`model.eval()`), and that you match categories correctly.
- **Result visualization:** It can help to occasionally save predicted masks overlayed on images for qualitative checks. Use `matplotlib` or torchvision's utils to draw boxes/masks. (TorchVision's tutorial includes a notebook for visualizing predictions on sample images.)

9. Experiment Tracking and Logging

Use experiment trackers to record hyperparameters and metrics over time:

- **TensorBoard:** PyTorch natively supports TensorBoard via `SummaryWriter`. As docs show, initialize `writer = SummaryWriter()` and log scalars with `writer.add_scalar(tag, value, step)` ¹⁷ ¹⁸. You can log training/validation loss, AP metrics, learning rate, etc. Then run `tensorboard --logdir=runs` to visualize. Example:

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter(log_dir="runs/exp1")
# inside training loop:
writer.add_scalar("Loss/train", train_loss, epoch)
writer.add_scalar("Loss/val", val_loss, epoch)
```

This is straightforward and doesn't require extra sign-up or internet.

- **Weights & Biases (wandb):** WandB provides a more powerful dashboard and collaboration. Integrate by calling `wandb.init()` at the start, logging config and metrics. For example, from the wandb docs:

```
import wandb
wandb.init(project="bubble-segmentation")
wandb.config = {"learning_rate": args.lr, "epochs": args.epochs,
               "batch_size": args.batch_size}
wandb.watch(model) # optional: track gradients
# In training loop:
wandb.log({"epoch": epoch, "loss": train_loss, "val_loss": val_loss})
```

This will upload metrics to your W&B account in real time ¹⁹. W&B also supports hyperparameter sweeps and visualizations. See their PyTorch tutorial for details ¹⁹.

- **MLflow:** MLflow can log experiments locally or remotely. Use `mlflow.log_param()` and `mlflow.log_metric()` in your training script. Example from the MLflow docs:

```
import mlflow
mlflow.log_param("lr", args.lr)
mlflow.log_param("batch_size", args.batch_size)
# in loop:
mlflow.log_metric("train_loss", train_loss, step=epoch)
mlflow.log_metric("val_loss", val_loss, step=epoch)
# at end:
mlflow.pytorch.log_model(model, "model")
```

MLflow's PyTorch guide suggests logging params/metrics at training start/end ²⁰. It also has a PyTorch "flavor" to save models. Use MLflow's UI or command-line to compare runs.

- **Others:** You could also use Neptune, Comet, TensorBoardX, etc. The above three (TB, W&B, MLflow) are most common and integrate well with PyTorch.

10. Modularity and Code Organization

Structure your code into reusable modules. For example:

- `dataset.py`: Define `BubbleDataset` and any transforms.

- `model.py` : Contain functions to build models (e.g. `get_mask_rcnn(num_classes)`). You can include multiple model constructors here.
- `train.py` : Main training script (parses args, loads data, builds model, training loop).
- `evaluate.py` : (Optional) Script to run evaluation on a saved checkpoint.
- `utils.py` : Common utilities (e.g., `collate_fn`, visualization helpers).
- `config.yaml` / `config.json` : Default hyperparameters, with option to override via CLI.

Using this structure, you can swap components easily. For instance, if you want to try Detectron2, you might write a new script (or extend `train.py`) that imports `detectron2.engine` instead of your Torch loop. Because your dataset is in COCO format, you could also use Detectron2's `build_detection_train_loader` with a COCO-style dataset ²¹.

Frameworks like **PyTorch Lightning** or **Catalyst** enforce modular design (Lightning separates data, model, and training logic) but are optional. The key is clear interfaces: e.g. your dataset always returns `(img, target)` and your model always expects that format. Then any model from torchvision, Detectron2, etc., can plug into the same data pipeline.

11. Example Projects and References

- **Mask R-CNN on Custom Data:** Arpit Aggarwal's GitHub demonstrates fine-tuning Mask R-CNN on the Penn-Fudan pedestrian dataset with PyTorch ⁷. The structure (dataset class, train/test scripts) can be a useful template.
- **TorchVision Tutorial:** The official PyTorch tutorial "Object Detection Finetuning" shows end-to-end Mask R-CNN on a custom dataset ⁴ ¹⁰. Its code (in [references/detection](#)) provides `train_one_epoch` and `evaluate` that you can reuse.
- **Detectron2:** Facebook's Detectron2 repo ¹⁴ and documentation have examples for custom datasets. The Detectron2 GitHub is at [facebookresearch/detectron2](#) ¹⁴. Roboflow's blog "How to Train Detectron2 for Custom Instance Segmentation" is also a helpful tutorial.
- **MMDetection:** OpenMMLab's [mmdetection](#) repo ¹⁵ is a highly modular toolbox. Its config tutorials show how to adjust models and data. The repo's "configs/mask_rcnn" folder has example config files.
- **LabelMe2COCO:** The [labelme2coco](#) project ³ can convert brush-based LabelMe JSONs to COCO format. It's a quick way to get your annotated brush labels into the needed format.
- **YOLOACT:** For a different model, see the [YOLOACT](#) repo by Daniel Bolya ¹⁶. It implements a fast real-time instance segmentation network in PyTorch.
- **W&B & MLflow:** Refer to Weights & Biases' PyTorch integration docs ¹⁹ and MLflow's PyTorch guide ²⁰ for code samples on logging.
- **TensorBoard:** PyTorch's recipe "How to use TensorBoard" shows logging usage ¹⁷ ¹⁸.
- **MMDetection Configs:** The MMDetection docs explain how to modify configs (via `--cfg-options`) for experiments ¹³. This is an advanced pattern if you adopt their framework.

Each of these resources contains code examples and patterns you can adapt. By combining lessons from them, you can build a robust, configurable pipeline. For instance, use the TorchVision tutorial's training loop, label conversion utilities like `labelme2coco`, and logging integration as shown in the W&B/MLflow docs.

Key Takeaways: Use COCO format for masks, leverage pre-trained Mask R-CNN from TorchVision as a starting point, manage hyperparameters via CLI/config, log experiments with tools like W&B or TensorBoard, and keep your code modular so you can swap in new models (Detectron2, YOLOACT, etc.) easily. The GitHub repositories and tutorials linked above provide concrete examples of each step.

- 1 7 GitHub - arp95/mask_rcnn_instance_segmentation: Instance Segmentation using Mask R-CNN on a Custom Dataset
https://github.com/arp95/mask_rcnn_instance_segmentation
- 2 4 5 6 9 10 TorchVision Object Detection Finetuning Tutorial — PyTorch Tutorials 2.7.0+cu126 documentation
https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html?highlight=maskrcnn_resnet50_fpn
- 3 GitHub - fcakyon/labelme2coco: A lightweight package for converting your labelme annotations into COCO object detection format.
<https://github.com/fcakyon/labelme2coco>
- 8 maskrcnn_resnet50_fpn — Torchvision main documentation
https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.maskrcnn_resnet50_fpn.html
- 11 12 Hyperparameters — PyTorch Lightning 1.6.5 documentation
<https://pytorch-lightning.readthedocs.io/en/1.6.5/common/hyperparameters.html>
- 13 Tutorial 1: Learn about Configs — MMDetection 2.27.0 documentation
<https://mmdetection.readthedocs.io/en/v2.27.0/tutorials/config.html>
- 14 GitHub - facebookresearch/detectron2: Detectron2 is a platform for object detection, segmentation and other visual recognition tasks.
<https://github.com/facebookresearch/detectron2>
- 15 GitHub - open-mmlab/mmdetection: OpenMMLab Detection Toolbox and Benchmark
<https://github.com/open-mmlab/mmdetection>
- 16 GitHub - dbolya/yolact: A simple, fully convolutional model for real-time instance segmentation.
<https://github.com/dbolya/yolact>
- 17 18 How to use TensorBoard with PyTorch — PyTorch Tutorials 2.7.0+cu126 documentation
https://docs.pytorch.org/tutorials/recipes/recipes/tensorboard_with_pytorch.html
- 19 PyTorch | Weights & Biases Documentation
<https://docs.wandb.ai/tutorials/pytorch/>
- 20 PyTorch within MLflow | MLflow
<https://mlflow.org/docs/latest/deep-learning/pytorch/guide>
- 21 How to Train Detectron2 for Custom Instance Segmentation
<https://blog.roboflow.com/detectron2-custom-instance-segmentation/>